TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN, ĐHQG-HCM

KHOA: KHOA HỌC MÁY TÍNH

LỚP: TTNT2025

BÁO CÁO STRING MATCHING

LỰA CHỌN THUẬT TOÁN TÌM KIẾM CHUỖI

Môn học: IT003.Q21.TTNT – Cấu trúc dữ liệu và giải thuật

Giảng viên hướng dẫn: ThS. Nguyễn Thanh Sơn

Thực hiện bởi Sinh Viên: Hoàng Lê Kim Lâm – MSSV: 25520974

Thời gian thực hiện: 10/03/2026 – 16/03/2026

Contents

TÌNH HUỐNG 4

1. Tình huống 1: MỘT MẪU TRÊN MỘT VĂN BẢN (Single Pattern – Single Text) 4

1.1 Giới thiệu vấn đề 4

1.2 Đặc tính dữ liệu cần tối ưu 4

1.3 Thuật toán đề xuất 1: Boyer–Moore 5

1.4 Thuật toán đề xuất 2: Two-Way String Matching 6

2. Tình huống 2: MỘT MẪU TRÊN NHIỀU VĂN BẢN (Single Pattern – Multiple Texts) 7

2.1 Giới thiệu vấn đề 7

2.2 Đặc tính dữ liệu cần tối ưu 7

2.3 Thuật toán đề xuất 1: KMP 7

2.4 Thuật toán đề xuất 2: Two-Way 8

2.5 Vì sao Two-Way dùng chung được cho cả trường hợp 1 và 2 8

3. Tình huống 3: NHIỀU MẪU TRÊN MỘT VĂN BẢN (Multiple Patterns – Single Text) 9

3.1 Giới thiệu vấn đề 9

3.2 Đặc tính dữ liệu cần tối ưu 9

3.3 Thuật toán đề xuất: Aho–Corasick 9

3.4 Cơ chế thuật toán và vì sao phù hợp 9

3.5 Worst-case 10

3.6 Ứng dụng thực tế 10

4. Tình huống 4: NHIỀU MẪU TRÊN NHIỀU VĂN BẢN (Multiple Patterns – Multiple Texts) 11

4.1 Giới thiệu vấn đề 11

4.2 Đặc tính dữ liệu cần tối ưu 11

4.3 Thuật toán đề xuất (Aho–Corasick – Pattern Indexing) 11

4.4 Cơ chế thuật toán và lý do phù hợp 12

4.5 Thuật toán đề xuất (Suffix Array – Text Indexing) 12

4.6 Cơ chế thuật toán và lý do phù hợp 12

4.7 So sánh với các thuật toán của các trường hợp trước 12

4.8 Worst-case **13**
4.9 Ứng dụng thực tế **13**
4.10 Kết luận **13**
Kết luận **13**
Thông tin code mô phỏng **14**

GIỚI THIỆU CHUNG

Bài toán String Matching là bài toán tìm một chuỗi mẫu trong một chuỗi văn bản. Cho một text T có độ dài nnn và một pattern P có độ dài mmm, ta cần tìm mọi vị trí i sao cho:

$$T[i..i+m-1] = P$$

Tuy nhiên, tùy theo số lượng pattern và số lượng text, bản chất tối ưu của bài toán thay đổi rất mạnh. Vì vậy không có một thuật toán duy nhất tối ưu cho tất cả tình huống. Có bốn trường hợp quan trọng: một pattern trên một text, một pattern trên nhiều text, nhiều pattern trên một text, và nhiều pattern trên nhiều text. Mỗi trường hợp sẽ có một chiến lược tối ưu khác nhau dựa trên đặc tính dữ liệu cần xử lý.

TÌNH HUỐNG

1. Tình huống 1: MỘT MẪU TRÊN MỘT VĂN BẢN (Single Pattern – Single Text)

1.1 Giới thiệu vấn đề

Trong bài toán tìm kiếm chuỗi cơ bản nhất, ta chỉ cần tìm một pattern P trong một text T. Đây là tình huống quen thuộc trong tìm từ khóa trong tài liệu, chức năng Find trong text editor, tìm chuỗi trong mã nguồn, hoặc cài đặt các hàm tìm substring trong thư viện chuẩn. Đặc điểm lớn nhất ở đây là chỉ có **một lần tìm kiếm duy nhất**. Vì vậy chi phí tiền xử lý pattern nếu có cũng không phải yếu tố quyết định. Điều cần tối ưu thực sự là số lần so sánh ký tự trên text. Nói cách khác, thuật toán phù hợp nhất là thuật toán có khả năng nhận biết mismatch sớm và bỏ qua được càng nhiều vị trí trong text càng tốt.

1.2 Đặc tính dữ liệu cần tối ưu

Trường hợp này có ba đặc điểm nổi bật. Thứ nhất, chỉ có một pattern nên không cần xây dựng cấu trúc dữ liệu phục vụ nhiều pattern. Thứ hai, text thường dài hơn pattern rất nhiều, tức là n≫m (n là length của text, m là length của pattern) . Thứ ba, đây là một lần tìm kiếm đơn lẻ nên không có lợi ích lớn từ việc đầu tư vào preprocessing quá nặng. Với đặc tính đó, thuật toán tốt nhất không phải là thuật toán quét tuần tự từng ký tự theo cách ngây thơ, mà là thuật toán có thể tận dụng mismatch để nhảy xa. Chính vì vậy, những thuật toán như Boyer–Moore và Two-Way nổi bật hơn cả.

1.3 Thuật toán đề xuất 1: Boyer–Moore

Boyer–Moore là một trong những thuật toán kinh điển và thực tế hiệu quả nhất cho bài toán một pattern trên một text. Ý tưởng chính của nó là so sánh pattern với text từ phải sang trái, thay vì từ trái sang phải như brute force. Thuật toán sử dụng hai heuristic quan trọng: Bad Character Rule và Good Suffix Rule. Với Bad Character Rule, nếu xảy ra mismatch tại một ký tự trong text, pattern được dịch sao cho ký tự đó khớp với lần xuất hiện gần nhất của nó trong pattern; nếu ký tự đó không có trong pattern thì pattern có thể được dịch toàn bộ độ dài m . Với Good Suffix Rule, nếu một suffix của pattern đã khớp nhưng mismatch xảy ra ở phía trước, thuật toán sẽ tìm vị trí khác của suffix này trong pattern để dịch pattern hợp lý hơn. Việc xây dựng bảng Bad Character cần duyệt toàn bộ pattern để lưu vị trí xuất hiện của từng ký tự, do đó có độ phức tạp O(m) với m là độ dài pattern. Bảng Good Suffix cũng được tính toán bằng cách phân tích các suffix của pattern, thường có chi phí O(m). rong thực tế, nhiều cài đặt đơn giản chỉ sử dụng **Bad Character heuristic** (biến thể Boyer–Moore–Horspool) vì vẫn đạt hiệu năng rất tốt và giúp giảm độ phức tạp của code. Trong báo cáo này, phần code minh họa sử dụng **Bad Character heuristic** để thể hiện ý tưởng chính của thuật toán.

Code minh họa : Boyer_Moore.py (link github ở cuối báo cáo)

Điểm cốt lõi khiến Boyer–Moore phù hợp là nó tối ưu cho tình huống chỉ tìm một lần duy nhất. Vì chỉ chạy một lần nên chi phí tiền xử lý pattern được "trả một lần" và sau đó thuật toán cố gắng giảm mạnh số ký tự text phải đọc. Trong dữ liệu thực tế, mismatch thường xảy ra khá sớm khi so sánh từ phải sang trái, nên pattern có thể nhảy nhiều bước mà không cần kiểm tra từng vị trí. Đây là lý do Boyer–Moore thường rất nhanh trong thực hành và có thể có hành vi gần sublinear trên nhiều tập dữ liệu. Về bản chất, nó không cố gắng duyệt mọi ký tự của text mà cố gắng **loại bỏ càng nhiều vị trí không có khả năng khớp càng tốt** hành vi trung bình của thuật toán thường được xem là gần:

O(n/m)

trong đó n là độ dài text

So sánh với các thuật toán khác:

- So sánh với brute force, Boyer–Moore mạnh hơn ở chỗ brute force luôn so sánh pattern tại từng vị trí của text và không tận dụng được thông tin mismatch.
- So sánh với KMP, Boyer–Moore lại khác ở triết lý: KMP bảo đảm tuyến tính nhưng vẫn phải đọc gần như toàn bộ text (khi mismatch xảy ra: con trỏ text i không lùi lại, pattern được dịch dựa trên LPS) ; Boyer–Moore thì cố nhảy xa để giảm số so sánh trung bình.

Worst-case của Boyer–Moore xảy ra khi text có nhiều ký tự lặp, pattern có cấu trúc bất lợi, hoặc các heuristic không tạo được bước nhảy lớn. Ví dụ, nếu text toàn ký tự lặp và

pattern gần giống nhưng sai ở cuối, thuật toán có thể chỉ dịch từng bước nhỏ và trở về độ phức tạp O(nm). Dù vậy, trong dữ liệu tự nhiên, worst-case này không phải lúc nào cũng xuất hiện, nên Boyer–Moore vẫn là một lựa chọn cực mạnh cho một lần tìm kiếm đơn lẻ.

Ứng dụng thực tế của Boyer–Moore thường gặp trong các công cụ tìm kiếm văn bản, text editor, các lệnh kiểu grep, và nhiều hệ thống search substring. Đây là nơi mà việc giảm số lần so sánh trung bình có ý nghĩa lớn hơn việc chỉ đảm bảo tuyến tính tuyệt đối.

1.4 Thuật toán đề xuất 2: Two-Way String Matching

Two-Way là thuật toán tìm kiếm chuỗi được đề xuất để cân bằng giữa hiệu năng thực tế và đảm bảo lý thuyết. Khác với Boyer–Moore tập trung mạnh vào các heuristic dịch chuyển theo vị trí mismatch, Two-Way dựa trên hai khái niệm của lý thuyết chuỗi: **critical factorization** và **periodicity**. Ý tưởng là tách pattern thành hai phần P=uv tại một vị trí đặc biệt gọi là critical position với độ phức tạp là O(m) (m là độ dài pattern). Sau đó thuật toán xác định chu kỳ của pattern, tức là khoảng cách nhỏ nhất p sao cho: P[i]=P[i+p] (độ phức tạp cũng là O(m))

Chu kỳ này phản ánh cấu trúc lặp nội tại của pattern. Sau khi phân tích pattern một lần, thuật toán sẽ so sánh phần phải trước, rồi mới kiểm tra phần trái nếu cần. Nếu mismatch xảy ra, Two-Way dịch pattern dựa trên chu kỳ và các tính chất đã chứng minh của phân tích này. Điều quan trọng là nó không cần quay lại kiểm tra lại nhiều ký tự text đã được loại trừ trước đó. (độ phức tạp bị chặn bởi O(n), n là độ dài text)

Code mô phỏng: Two_way.py (link github ở cuối báo cáo)

Cơ chế này phù hợp với trường hợp một pattern trên một text vì thuật toán khai thác trực tiếp cấu trúc của pattern để kiểm soát bước dịch. Nó có hai ưu điểm đồng thời: thứ nhất, nó phát hiện mismatch sớm bằng cách so sánh một phần pattern trước; thứ hai, nó vẫn có khả năng nhảy xa nhờ sử dụng chu kỳ. Chính vì vậy Two-Way có hiệu năng thực tế rất tốt nhưng vẫn đảm bảo độ phức tạp tuyến tính trong mọi trường hợp. Đây là điểm khác biệt lớn so với Boyer–Moore: BM có thể rất nhanh trung bình nhưng worst-case xấu hơn; Two-Way thì giữ được tốc độ thực tế tốt mà không đánh đổi worst-case.

So với các thuật toán khác:

- So với KMP, Two-Way không đơn thuần chỉ dựa trên prefix-suffix như LPS mà còn tận dụng cấu trúc chu kỳ sâu hơn của pattern.
- So với Boyer–Moore, Two-Way không phụ thuộc nặng vào đặc điểm ngẫu nhiên của text hay alphabet; nó có tính ổn định tốt hơn.

→ Vì thế Two-Way được nhiều thư viện hệ thống chọn làm thuật toán mặc định cho hàm tìm substring. Trong các thư viện chuẩn, mục tiêu không phải là tối ưu riêng cho một tập dữ liệu đặc biệt mà là bảo đảm tốc độ ổn định, bộ nhớ nhỏ và worst-case tuyến tính.

Worst-case của Two-Way vẫn là O(n+m). Đây là điểm rất mạnh: cho dù text có cấu trúc xấu thế nào, thuật toán vẫn không bị suy giảm như Boyer–Moore. Về mặt thực tế, Two-Way xuất hiện trong các triển khai hàm tìm chuỗi như `strstr()` trong các thư viện hệ thống. Điều này cho thấy nó không chỉ là một thuật toán đẹp về lý thuyết mà còn là một giải pháp đã được kiểm chứng trong phần mềm thật.

2. Tình huống 2: MỘT MẪU TRÊN NHIỀU VĂN BẢN (Single Pattern – Multiple Texts)

2.1 Giới thiệu vấn đề

Trong trường hợp này, ta có một pattern cố định P nhưng cần tìm nó trong nhiều văn bản khác nhau T1,T2,...,Tk. Đây là bài toán phổ biến trong giám sát log, phân tích nhiều file văn bản, tìm một gene trong nhiều bộ gen, hoặc quét một chuỗi cảnh báo trong hàng loạt dữ liệu. Điểm bản chất ở đây là pattern **không thay đổi**, nhưng việc tìm kiếm được lặp lại nhiều lần. Nói cách khác, bài toán không còn xoay quanh "tìm một lần" mà xoay quanh "tận dụng lại thông tin đã học từ pattern để dùng cho nhiều lần tìm kiếm".

2.2 Đặc tính dữ liệu cần tối ưu

So với trường hợp 1, sự khác biệt lớn nhất nằm ở chỗ pattern được tái sử dụng liên tục. Điều này làm cho chi phí preprocessing pattern trở nên rất đáng giá. Nếu ta phải tìm cùng một pattern trong hàng trăm hay hàng nghìn văn bản, việc tính lại các cấu trúc hỗ trợ cho pattern mỗi lần là lãng phí. Vì vậy thuật toán lý tưởng phải có hai tính chất: một là tách rõ bước phân tích pattern khỏi bước tìm kiếm; hai là sau khi phân tích xong thì có thể tái sử dụng kết quả trên mọi text. Đây là lý do KMP và Two-Way trở thành hai thuật toán rất phù hợp.

2.3 Thuật toán đề xuất 1: KMP

KMP xây dựng mảng LPS (Longest Prefix Suffix), trong đó mỗi giá trị cho biết độ dài prefix dài nhất của pattern cũng là suffix của đoạn pattern đang xét. Mảng này cho phép thuật toán biết rằng khi mismatch xảy ra, không cần quay lại đầu pattern mà có thể nhảy đến vị trí phù hợp trong pattern. Điều này giúp tránh so sánh lặp lại các ký tự đã được xác nhận.

Điểm quan trọng là mảng LPS chỉ phụ thuộc vào pattern. Vì pattern cố định, ta chỉ cần tính LPS **một lần duy nhất** với chi phí O(m) Sau đó mỗi văn bản chỉ cần duyệt một lần với chi phí O(|Ti|) Tổng chi phí cho toàn bộ tập văn bản là: $O(m+\sum|T_i|)$

Đây là một kết quả rất phù hợp với bài toán nhiều văn bản. Trong thực tế, nếu pattern được dùng lặp lại rất nhiều lần, hiệu quả cộng dồn từ việc chỉ tiền xử lý một lần là cực kỳ lớn. Chính vì vậy KMP thường được chọn trong các hệ thống cần độ ổn định cao và muốn tránh phụ thuộc vào các heuristic trung bình kiểu Boyer–Moore. KMP không nhảy xa như BM, nhưng nó bảo đảm rằng mọi text đều xử lý tuyến tính, không có tình huống bất lợi làm chi phí tăng đột biến.

So với Boyer–Moore, KMP yếu hơn về khả năng bỏ qua vị trí trong một lần tìm kiếm, nhưng mạnh hơn ở tính ổn định và khả năng tái sử dụng preprocessing. Với nhiều văn bản,

điều cần nhất không phải là một heuristic "nhảy xa nhất có thể" mà là một cấu trúc hỗ trợ ổn định để dùng lại nhiều lần. Đây chính là chỗ KMP phát huy vai trò.

Worst-case của KMP vẫn là O(|Ti|+m) cho mỗi văn bản, nên tổng chi phí luôn tuyến tính theo tổng độ dài toàn bộ dữ liệu. KMP không phụ thuộc vào việc text có cấu trúc xấu hay không, do đó nó rất phù hợp cho hệ thống cần hiệu năng dự đoán được. Ứng dụng của KMP thấy rõ trong các hệ thống quét log, phân tích mã nguồn và bioinformatics, nơi cùng một mẫu gene hoặc cùng một chuỗi cảnh báo được kiểm tra lặp lại trên nhiều nguồn dữ liệu.

Code mô phỏng KMP: KMP.py (link github ở cuối báo cáo)

2.4 Thuật toán đề xuất 2: Two-Way

Two-Way cũng rất phù hợp cho trường hợp một pattern trên nhiều text, nhưng theo một cách hơi khác KMP. Giống như ở trường hợp 1, Two-Way phân tích pattern thành hai phần dựa trên critical factorization và chu kỳ của pattern. Bước phân tích này chỉ cần thực hiện một lần, nên có thể tái sử dụng cho tất cả văn bản tiếp theo. Khi tìm trên mỗi text, thuật toán sử dụng kết quả phân tích để điều khiển bước dịch pattern, đồng thời vẫn giữ được độ phức tạp tuyến tính.

Điểm mạnh của Two-Way trong bối cảnh nhiều văn bản là nó kết hợp hai lợi ích cùng lúc. Một mặt, nó có preprocessing của pattern có thể tái sử dụng giống KMP. Mặt khác, nó vẫn có khả năng bỏ qua tốt hơn so với nhiều cách duyệt tuần tự thông thường vì nó dựa trên chu kỳ của pattern để quyết định bước nhảy. Nói cách khác, Two-Way không chỉ là một thuật toán tuyến tính ổn định mà còn là một thuật toán tuyến tính có hiệu năng thực tế rất tốt khi pattern được sử dụng lặp lại nhiều lần. Điều này khiến nó trở thành một lựa chọn "dùng chung" rất mạnh cho cả trường hợp 1 và 2.

Khác với KMP, Two-Way không dựa hoàn toàn trên LPS mà khai thác sâu hơn tính chu kỳ của chuỗi. Khác với Boyer–Moore, nó không bị lệ thuộc quá nhiều vào việc text có "hợp" heuristic hay không. Vì vậy khi viết báo cáo, có thể xem Two-Way như một thuật toán tối ưu mang tính tổng quát, còn KMP là lựa chọn đơn giản, trực tiếp và rất ổn định. Trong hệ thống thư viện, Two-Way thường được chọn khi người ta muốn một hàm tìm substring có tính tổng quát cao, ít rủi ro worst-case và vẫn đủ nhanh trên thực tế.

Worst-case của Two-Way vẫn là O(|Ti|+m) cho mỗi văn bản. Vì vậy tổng chi phí vẫn tuyến tính theo tổng kích thước toàn bộ tập văn bản. Điều này rất quan trọng vì trong bối cảnh xử lý nhiều text, sự ổn định có giá trị hơn việc nhảy xa trong một vài trường hợp đặc biệt. Ứng dụng của Two-Way thể hiện rõ trong các thư viện hệ thống khi triển khai strstr() và các hàm substring tương tự.

2.5 Vì sao Two-Way dùng chung được cho cả trường hợp 1 và 2

Two-Way được xem là "dùng chung" cho hai trường hợp vì cơ chế tối ưu của nó nằm ở chính **cấu trúc của pattern**, không phải ở việc text có bao nhiêu hay chỉ tìm một lần hay nhiều lần. Trong trường hợp 1, Two-Way tận dụng chu kỳ để giảm số phép so sánh và phát hiện mismatch sớm trên một text duy nhất. Trong trường hợp 2, phần phân tích pattern một

lần có thể tái sử dụng cho nhiều text, nên Two-Way cũng thích hợp như KMP. Nói cách khác, Two-Way không tối ưu riêng cho "một text" hay "nhiều text", mà tối ưu cho việc **khai thác pattern một cách có hệ thống**, nên nó đứng ở vị trí giữa KMP và Boyer–Moore: vừa giữ tính ổn định tuyến tính, vừa có hiệu năng thực tế tốt.

3. Tình huống 3: NHIỀU MẪU TRÊN MỘT VĂN BẢN (Multiple Patterns – Single Text)

3.1 Giới thiệu vấn đề

Trong trường hợp này, ta không còn tìm một pattern duy nhất nữa mà cần tìm **nhiều pattern khác nhau** trong cùng một text. Ví dụ điển hình là quét nhiều chữ ký virus trong một file, tìm nhiều từ khóa nhạy cảm trong một đoạn chat, hoặc dò nhiều mô típ DNA trong một chuỗi gene. Đây là bài toán mà số lượng pattern có thể rất lớn. Đặc điểm cốt lõi là **text chỉ cần quét một lần**, còn pattern thì nhiều. Vì vậy chiến lược tối ưu không còn là tối ưu một pattern đơn lẻ nữa mà là **gộp toàn bộ pattern vào một cấu trúc duy nhất để xử lý đồng thời**.

3.2 Đặc tính dữ liệu cần tối ưu

So với hai trường hợp trước, trường hợp này thay đổi hoàn toàn về bản chất tối ưu. Ở trường hợp 1 và 2, ta tối ưu việc xử lý một pattern; ở đây, ta phải tối ưu việc xử lý một **tập pattern**. Nếu ta dùng KMP hoặc Boyer–Moore cho từng pattern riêng lẻ, text sẽ phải bị quét đi quét lại rất nhiều lần, khiến chi phí tăng thành: $O(k \cdot n)$

với k là số lượng pattern. Khi k lớn, chi phí này trở nên không thể chấp nhận được. Chính vì vậy bài toán cần một thuật toán có khả năng **kiểm tra tất cả pattern cùng lúc trong một lần quét text**. Đây là điểm mà các thuật toán của trường hợp 1 và 2 không còn phù hợp nữa.

3.3 Thuật toán đề xuất: Aho–Corasick

Aho–Corasick là thuật toán chuẩn cho bài toán nhiều pattern trên một text. Ý tưởng của nó là xây dựng một **trie** chứa toàn bộ pattern, sau đó bổ sung **failure links** để tạo thành một automaton. Trie giúp chia sẻ các prefix chung giữa những pattern, còn failure links giúp xử lý mismatch bằng cách quay về prefix dài nhất có khả năng tiếp tục khớp, tương tự nguyên lý của KMP nhưng mở rộng cho nhiều pattern.

Cấu trúc này biến tập pattern thành một máy trạng thái duy nhất. Khi quét text, mỗi ký tự chỉ được xử lý một lần. Nếu đang ở một trạng thái nào đó mà ký tự tiếp theo không phù hợp, automaton không quay lại đầu mà sử dụng failure links để chuyển sang trạng thái hợp lý hơn. Khi đi tới một node kết thúc pattern, thuật toán báo match. Nếu nhiều pattern kết thúc tại cùng một trạng thái, tất cả đều được phát hiện cùng lúc.

3.4 Cơ chế thuật toán và vì sao phù hợp

Đây là điểm quan trọng nhất của Aho–Corasick. Bản chất của bài toán nhiều pattern trên một text là có rất nhiều pattern thường chia sẻ prefix chung. Ví dụ trong bộ pattern `"he"`, `"she"`, `"his"`, `"hers"`, ta thấy các prefix như `"h"`, `"he"`, `"sh"` được dùng lại nhiều lần. Nếu

chạy KMP cho từng pattern, phần so khớp prefix này sẽ bị lặp đi lặp lại rất nhiều. Aho–Corasick giải quyết chính xác vấn đề này bằng trie: Mỗi cạnh của trie tương ứng với một ký tự, và mỗi node biểu diễn một prefix của một hoặc nhiều pattern. Nhờ cấu trúc này, các prefix chung giữa các pattern chỉ được lưu một lần .Vì vậy chi phí của bước này là: O(M) (M là tổng độ dài tất cả pattern). Failure link của một node trỏ tới node biểu diễn **prefix dài nhất khác chính nó nhưng cũng là suffix của chuỗi hiện tại**. Failure links thường được xây bằng **BFS trên trie**. Mỗi cạnh và mỗi node được xử lý một lần trong quá trình này. Do đó chi phí của bước xây failure links cũng là:O(M). Kết quả là text được quét đúng một lần, nhưng vẫn phát hiện được tất cả pattern cùng lúc.Sau khi automaton hoàn chỉnh, thuật toán bắt đầu quét text. Khi đọc mỗi ký tự của text, automaton thực hiện một trong hai hành động đi theo cạnh tương ứng nếu ký tự phù hợp, hoặc theo failure link để tìm trạng thái hợp lệ tiếp theo. Nhờ failure links, con trỏ trên text **không bao giờ lùi lại**, và mỗi ký tự của text được xử lý đúng một lần. Nếu độ dài text là: n thì chi phí của giai đoạn này là O(n) .Ngoài ra, mỗi lần gặp node kết thúc pattern, thuật toán báo match. Nếu có tổng cộng kkk lần match thì chi phí xuất kết quả là: O(k)

Đây là lý do Aho–Corasick phù hợp hơn hẳn KMP, Boyer–Moore hay Two-Way trong tình huống này. KMP, Boyer–Moore và Two-Way đều là các thuật toán sinh ra để giải quyết **một pattern**. Nếu sử dụng chúng lặp lại cho nhiều pattern, ta đang bỏ lỡ cơ hội hợp nhất các pattern có cấu trúc chung. Aho–Corasick thì ngược lại: nó biến toàn bộ bộ pattern thành một cấu trúc thống nhất, nên tối ưu được đúng đặc tính dữ liệu của bài toán.

So với trường hợp 1 và 2, ở đây thuật toán không tối ưu text đơn lẻ cũng không tối ưu việc reuse một pattern đơn lẻ, mà tối ưu **việc reuse cấu trúc giữa các pattern**. Đây là sự khác biệt bản chất. Chính vì vậy việc dùng lại thuật toán của trường hợp 1 hoặc 2 cho trường hợp này là không hiệu quả.

3.5 Worst-case

Aho–Corasick có preprocessing là xây trie và failure links, với chi phí phụ thuộc vào tổng độ dài của toàn bộ pattern. Sau đó quá trình quét text là tuyến tính theo độ dài text cộng với số lượng kết quả tìm thấy. Tức là tổng thể vẫn đạt mức gần tuyến tính theo dữ liệu đầu vào. Đây là một điểm cực kỳ quan trọng: dù số lượng pattern lớn, thuật toán vẫn không bị tăng thành O(k·n) như cách chạy KMP/BM riêng lẻ. (k là số pattern)

3.6 Ứng dụng thực tế

Aho–Corasick được dùng rất rộng rãi trong các hệ thống bảo mật và lọc nội dung. Antivirus dùng nó để quét nhiều chữ ký virus trong một file; IDS/IPS dùng nó để dò nhiều mẫu tấn công mạng; spam filter dùng nó để tìm nhiều từ khóa cấm trong email. Điểm chung của các hệ thống này là cần tìm rất nhiều mẫu khác nhau trong một luồng dữ liệu duy nhất và cần làm điều đó nhanh, ổn định, không quét lại nhiều lần.

Code mô phỏng: Aho_Corasick.py (link github ở cuối báo cáo)

4. Tình huống 4: NHIỀU MẪU TRÊN NHIỀU VĂN BẢN (Multiple Patterns – Multiple Texts)

4.1 Giới thiệu vấn đề

Trường hợp thứ tư là tình huống tổng quát nhất của bài toán string matching, khi cả số lượng pattern và số lượng text đều lớn. Ta có một tập mẫu P={P1,P2,...,Pm} và một tập văn bản T={T1,T2,...,Tk}. Bài toán đặt ra là tìm tất cả các vị trí xuất hiện của mọi pattern trong toàn bộ tập text. Đây là tình huống thường gặp trong các hệ thống xử lý dữ liệu quy mô lớn như hệ thống tìm kiếm văn bản, hệ thống phân tích log, hệ thống phát hiện tấn công mạng hoặc các ứng dụng sinh học tính toán. So với ba trường hợp trước, ở đây cả hai chiều dữ liệu đều lớn nên việc áp dụng trực tiếp các thuật toán như KMP, Boyer–Moore hay Two-Way cho từng cặp pattern–text riêng lẻ sẽ dẫn đến chi phí tính toán rất cao. Nếu ta chạy một thuật toán tìm kiếm cho mỗi pattern trên mỗi text thì chi phí tổng thể có thể lên đến O(m·k·n) với nnn là độ dài trung bình của text. Khi dữ liệu lớn, cách làm này trở nên không khả thi. Vì vậy trường hợp 4 không chỉ đơn giản là chọn một thuật toán tìm kiếm tối ưu cho một lần so khớp, mà cần lựa chọn một chiến lược xử lý dữ liệu cho phép tái sử dụng thông tin đã tiền xử lý. Hai chiến lược chính thường được sử dụng là **pattern indexing** và **text indexing**, tương ứng với hai thuật toán tiêu biểu là Aho–Corasick và suffix array.

4.2 Đặc tính dữ liệu cần tối ưu

Trong trường hợp này, việc lựa chọn thuật toán phụ thuộc vào đặc tính của dữ liệu. Nếu tập pattern gần như cố định nhưng số lượng text rất lớn, ta cần một phương pháp cho phép xây dựng cấu trúc dữ liệu từ pattern một lần và sau đó áp dụng cho nhiều text khác nhau. Ngược lại, nếu tập text lớn nhưng tương đối ổn định còn pattern thay đổi thường xuyên, thì việc xây dựng một chỉ mục cho text sẽ hiệu quả hơn. Vì vậy trường hợp 4 thực chất có hai kịch bản con. Trong kịch bản thứ nhất, pattern ổn định và text thay đổi nhiều; trong kịch bản thứ hai, text ổn định và pattern thay đổi nhiều. Hai kịch bản này dẫn đến hai hướng tiếp cận thuật toán khác nhau.

4.3 Thuật toán đề xuất (Aho–Corasick – Pattern Indexing)

Trong kịch bản mà tập pattern ổn định nhưng có nhiều text cần quét, Aho–Corasick là thuật toán phù hợp. Thuật toán này xây dựng một automaton từ toàn bộ tập pattern bằng cách kết hợp cấu trúc trie với các failure link tương tự cơ chế LPS trong KMP. Đầu tiên, tất cả pattern được chèn vào một trie, trong đó mỗi đường đi từ gốc đến một nút đại diện cho một tiền tố của pattern. Sau đó thuật toán bổ sung các failure link để xác định điểm quay lui hợp lệ khi xảy ra mismatch trong quá trình quét text. Sau khi hoàn thành bước tiền xử lý, cấu trúc trie cùng các failure link tạo thành một automaton hữu hạn. Khi quét text, automaton sẽ chuyển trạng thái theo từng ký tự và tự động phát hiện mọi pattern xuất hiện. Điều quan trọng là automaton này chỉ cần được xây dựng một lần từ tập pattern, sau đó có thể được sử dụng lại cho mọi text. Độ phức tạp xây dựng automaton là O(tổng độ dài pattern), và việc quét mỗi text có chi phí O(n + {số lần khớp}) Nếu có nhiều text, tổng chi phí tìm kiếm sẽ là O(∑|Ti|)

tức tuyến tính theo tổng độ dài của toàn bộ văn bản. Nhờ cơ chế này, Aho–Corasick có thể phát hiện đồng thời tất cả pattern chỉ bằng một lần duyệt text.

4.4 Cơ chế thuật toán và lý do phù hợp

Cơ chế của Aho–Corasick dựa trên ý tưởng gộp toàn bộ pattern vào một automaton duy nhất. Trong khi KMP hoặc Boyer–Moore chỉ xử lý một pattern tại một thời điểm, Aho–Corasick cho phép kiểm tra tất cả pattern đồng thời. Điều này đặc biệt quan trọng khi số lượng pattern lớn. Khi automaton được xây dựng, quá trình tìm kiếm trở thành một quá trình duyệt trạng thái tuyến tính theo text. Mỗi ký tự của text được đọc đúng một lần và automaton quyết định chuyển sang trạng thái nào dựa trên các cạnh trie và failure link. Nhờ vậy thuật toán tránh được việc lặp lại các phép so sánh không cần thiết. Trong trường hợp 4, khi pattern gần như cố định (ví dụ danh sách chữ ký virus hoặc chữ ký tấn công mạng), việc xây automaton một lần rồi sử dụng lại trên nhiều text giúp giảm đáng kể chi phí tính toán.

Code mô phỏng: Aho_Corasick_TH4.py (link github ở cuối báo cáo)

4.5 Thuật toán đề xuất (Suffix Array – Text Indexing)

Trong kịch bản ngược lại, khi tập text lớn và ổn định còn pattern thay đổi liên tục, chiến lược hiệu quả hơn là xây dựng một chỉ mục cho text. Một cấu trúc dữ liệu phổ biến cho mục đích này là suffix array. Suffix array lưu tất cả các suffix của text theo thứ tự từ điển. Giả sử ta có gộp tất cả text T và chúng có độ dài N tổng cộng. Ta liệt kê tất cả các suffix của tổng đó và sắp xếp chúng theo thứ tự từ điển. Mảng lưu vị trí bắt đầu của các suffix đã sắp xếp chính là suffix array. Khi cần tìm một pattern P độ dài m, ta có thể sử dụng binary search trên suffix array để xác định xem P có phải là tiền tố của một suffix nào đó hay không. Điều này biến bài toán tìm kiếm pattern thành một bài toán tra cứu trên một cấu trúc dữ liệu đã được xây dựng trước. Độ phức tạp xây dựng suffix array thường là (NlogN) hoặc O(N) với các thuật toán hiện đại. Sau khi index được xây dựng, mỗi truy vấn pattern có thể được trả lời trong thời gian O(mlogN).

4.6 Cơ chế thuật toán và lý do phù hợp

Suffix array phù hợp với trường hợp 4 khi phía text ổn định nhưng có nhiều truy vấn pattern. Thay vì quét lại toàn bộ text cho mỗi truy vấn như trong KMP hoặc Boyer–Moore, suffix array cho phép truy vấn trực tiếp trên cấu trúc đã được sắp xếp. Điều này giúp giảm đáng kể chi phí tìm kiếm khi số lượng truy vấn lớn. Điểm khác biệt quan trọng giữa suffix array và Aho–Corasick là vị trí của chỉ mục. Aho–Corasick xây chỉ mục cho pattern, trong khi suffix array xây chỉ mục cho text. Việc lựa chọn phương pháp nào phụ thuộc vào phía dữ liệu nào ổn định hơn và được sử dụng lặp lại nhiều lần.

4.7 So sánh với các thuật toán của các trường hợp trước

Các thuật toán của ba trường hợp trước không hoàn toàn phù hợp với trường hợp 4. Boyer–Moore và KMP chỉ xử lý một pattern tại một thời điểm, vì vậy khi có nhiều pattern và nhiều text chúng sẽ phải chạy lặp lại nhiều lần. Two-Way đảm bảo độ phức tạp tuyến tính cho một lần tìm kiếm nhưng cũng không giải quyết được vấn đề nhiều pattern. Aho–Corasick giải quyết tốt bài toán nhiều pattern trên một text, nhưng nếu text cũng thay đổi liên tục thì việc

quét lại toàn bộ text nhiều lần vẫn có thể tốn kém. Vì vậy trong trường hợp 4, cách tiếp cận hiệu quả hơn thường là sử dụng cấu trúc chỉ mục để tái sử dụng kết quả tiền xử lý.

Code Mô phỏng: Index_Suffix_Array.py (link github ở cuối báo cáo)

4.8 Worst-case

Đối với Aho–Corasick, độ phức tạp trong trường hợp xấu nhất vẫn là tuyến tính theo tổng kích thước dữ liệu, tức O(pattern length+∑|Ti|). Điều này đảm bảo rằng mỗi ký tự của text chỉ được đọc một lần. Đối với suffix array, chi phí xây dựng chỉ mục là O(NlogN) với N là tổng độ dài các text, nhưng sau đó mỗi truy vấn pattern chỉ mất O(mlogN) (m là độ dài pattern đó). Chi phí lớn được dồn vào bước preprocessing để đổi lại khả năng truy vấn nhanh.

4.9 Ứng dụng thực tế

Trong thực tế, các hệ thống phát hiện xâm nhập mạng và phần mềm antivirus thường sử dụng Aho–Corasick để quét hàng nghìn chữ ký trên luồng dữ liệu lớn. Ngược lại, các hệ thống tìm kiếm văn bản và cơ sở dữ liệu sinh học thường sử dụng các cấu trúc chỉ mục như suffix array hoặc suffix tree để hỗ trợ truy vấn nhanh trên các bộ dữ liệu rất lớn.

4.10 Kết luận

Trường hợp nhiều pattern trên nhiều text không có một thuật toán duy nhất tối ưu cho mọi tình huống. Nếu tập pattern ổn định và cần quét nhiều text, Aho–Corasick là lựa chọn hiệu quả nhờ khả năng gộp nhiều pattern vào một automaton duy nhất. Nếu tập text ổn định và có nhiều truy vấn pattern, các cấu trúc indexing như suffix array sẽ phù hợp hơn vì cho phép tra cứu trực tiếp trên dữ liệu đã được lập chỉ mục. Hai chiến lược này đại diện cho hai hướng tiếp cận khác nhau trong bài toán string matching: **pattern indexing** và **text indexing**.

Kết luận

Qua bốn trường hợp của bài toán **string matching**, có thể thấy rằng hiệu quả của thuật toán phụ thuộc mạnh vào **cấu trúc dữ liệu giữa pattern và text**. Mỗi thuật toán được thiết kế để tối ưu một tình huống cụ thể thay vì áp dụng tốt cho mọi trường hợp.

Trong **trường hợp một pattern trên một text**, mục tiêu là giảm số phép so sánh khi tìm kiếm. Các thuật toán như **Boyer–Moore** và **Two–Way** tận dụng cấu trúc của pattern để **dịch chuyển pattern xa khi xảy ra mismatch**, nhờ đó giảm đáng kể số lần so khớp trong thực tế. Boyer–Moore thường cho hiệu năng trung bình rất cao, trong khi Two–Way bảo đảm độ phức tạp tuyến tính O(n+m) và được sử dụng trong nhiều thư viện chuẩn.

Trong **trường hợp một pattern trên nhiều text**, pattern được tái sử dụng nhiều lần. Thuật toán **KMP** phù hợp vì nó tiền xử lý pattern một lần với chi phí O(m) để xây

dựng mảng LPS, sau đó mỗi văn bản chỉ cần quét tuyến tính O(n). Nhờ đó tổng chi phí cho nhiều văn bản gần bằng O(m+∑ni) và hiệu năng ổn định hơn so với các thuật toán phụ thuộc mạnh vào cấu trúc text.

Trong **trường hợp nhiều pattern trên một text**, vấn đề chính là tránh việc quét text nhiều lần cho từng pattern. Thuật toán **Aho–Corasick** giải quyết bằng cách xây dựng một **automaton từ toàn bộ tập pattern**, cho phép tìm tất cả pattern chỉ trong một lần duyệt text. Sau khi xây automaton với chi phí O(L) (tổng độ dài pattern), quá trình tìm kiếm chỉ cần O(n)

Trong **trường hợp nhiều pattern trên nhiều text**, có hai chiến lược. Nếu tập **pattern cố định nhưng có nhiều text cần quét**, ta có thể xây automaton **Aho–Corasick** một lần rồi áp dụng cho tất cả các văn bản. Ngược lại, nếu **text ổn định nhưng có rất nhiều truy vấn pattern khác nhau**, thì cách hiệu quả hơn là xây dựng **chỉ mục cho text**, ví dụ bằng **suffix array** hoặc **suffix tree**. Sau khi xây index với chi phí khoảng O(NlogN), mỗi truy vấn pattern có thể được trả lời nhanh trong khoảng O(mlogN) mà không cần quét lại toàn bộ dữ liệu.

Tóm lại, các thuật toán string matching tối ưu cho những đặc tính dữ liệu khác nhau: **Boyer–Moore và Two–Way** tối ưu cho tìm kiếm đơn lẻ, **KMP** phù hợp khi pattern được tái sử dụng nhiều lần, **Aho–Corasick** xử lý hiệu quả nhiều pattern cùng lúc, còn **suffix array/indexing** thích hợp cho hệ thống có nhiều truy vấn trên tập dữ liệu lớn. Việc lựa chọn thuật toán vì vậy phải dựa trên **đặc tính của dữ liệu và mô hình sử dụng của bài toán**.

Thông tin code mô phỏng

Link github: https://github.com/HoangLeKimLam/String-matching